1. Chained-S3 on N83, m = 9: parallel build of the depth-5 fingerprint set

Run `n83_chain_sat_parbuild_20261101`, 2026-11-01 22:58:33Z – 23:49:48Z. Protocol `PROTOCOL.md` frozen at `27ef5fa874` before the run (user: "now parallelize the depth-5 build for m=9"). Driver `examples/n83_chain_sat_bench.rs`; native analysis `examples/n83_chain_sat_analyze.rs` ⟶ `ANALYSIS.json` (warmup lines and query cells), `table.tsv`; graph by `plot.py` from `ANALYSIS.json` only. Host: one 4-vCPU x86-64 cloud VM, L0; four processes S1, P1, S2, P2, one at a time. Follow-up to 3247: m = 9 queries take ≈ 14 s on 4 threads, but the one-off single-threaded build of the depth-5 set took 1,023 s.

Curve. `icv1-f2m83-t6151469093347-cdcc5432`; base: Frobenius union of span(1, z), 167 points. Each process: 3 public + 5 planted m = 9 targets (seed 20261031), 300,000 conflicts and 120 s per query.

1.1. What changed (`ChainS3Tables::parallel_build`)

`chain_s3_build_fingerprints` splits the depth-d enumeration on the multiset's first index. The multisets whose smallest index is i number exactly $C(n-i+d-2, d-1)$, so every worker fills a disjoint slice of one array of the exact total size ($C(171,5) \approx 1.15 \cdot 10^9$ entries at $d=5$) with no per-worker vectors to merge; the slices are compacted over the identity sums, and the array is sorted in parallel, deduplicated and indexed as before. The resulting set equals the sequential one (unit test `n83_parallel_build_matches_sequential`, depths 1–3).

1.2. Results (measured, wall seconds)

depth	fingerprints (all 4 processes)	S1, S2 (1 thread)	P1, P2 (4 threads)	ratio of means
5	372,406,144	1,006.8; 993.4	287.7; 283.3	3.50×
4	12,976,303	27.0; 25.2	7.5; 7.9	3.4×
3	347,688	0.65; 0.56	0.17; 0.18	—

1. **Soundness.** Identical fingerprint counts at every depth in all four processes; every process decides all 8 queries (3/3 public refuted, 5/5 planted verified; 32 records, 0 soundness flags), the verdicts of 3247
2. **Build time.** Both parallel depth-5 builds (287.7 s, 283.3 s) are faster than both sequential ones (1,006.8 s, 993.4 s): 1,000.1 s ⟶ 285.5 s mean, 3.50× on 4 threads. Improvement under the frozen rule.
3. Query times are unchanged by the build (median ≈ 13.4–13.6 s in both arms, as expected: the queries already ran on 4 threads).
4. A cold m = 9 process on this host now spends ≈ 293 s building depths 2–5 and ≈ 14 s per query, against ≈ 1,026 s and ≈ 14 s before.

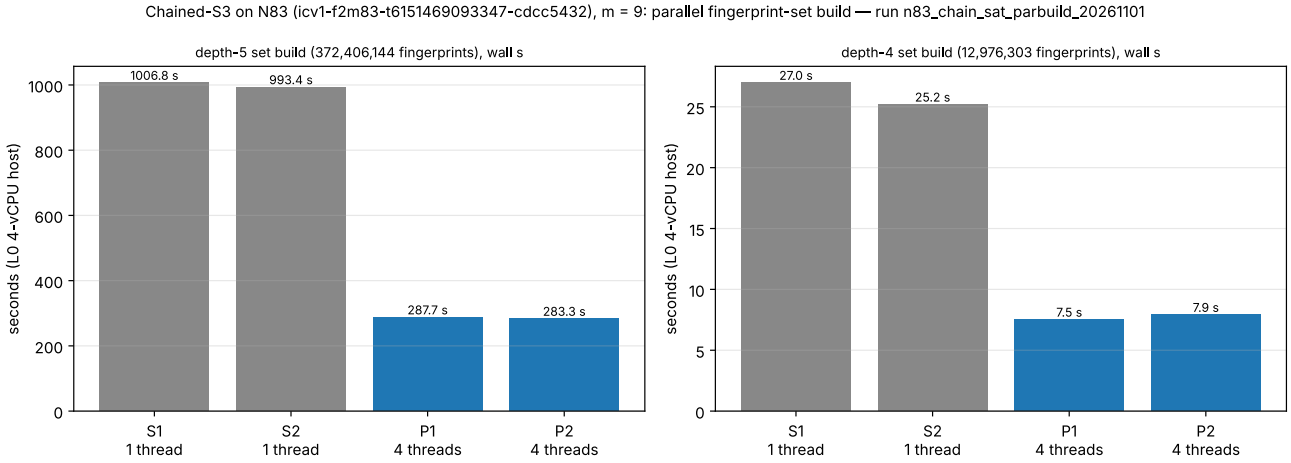


Figure 1: Wall seconds of the depth-5 and depth-4 builds per process (S = sequential build, P = parallel build).

1.3. What this is and is not

- A wall-clock speed-up on 4 vCPUs, not less work: the same $1.15 \cdot 10^9$ point additions and an equal-size sort. Peak memory was not measured (the array is allocated once at its exact size, as before).
- Decomposition stage only: no relation collection, no DLP, no leaderboard or `benchmarks/` row (validation gate). Matched rho on this curve: 1.51 s mean online (`research/n83_msweep_20261024`).

1.4. Limits

Two processes per arm on one L0 host; wall time only. EKS (AGENTS.md §9a) not used: not an ecbench method, no aws CLI.